PantryPilot Configuration Management Report

Prepared by	Zixuan Liang
Course	CISC 594: Software Testing Principles and Techniques
Instructor	Dr. Khalid Lateef
Project	PantryPilot - Smart Meal and Grocery Planner
Submission	HU14 Final Project: Configuration Management Report
Date	August 4, 2026

Executive Summary

This report documents how configuration management was applied to PantryPilot, the semester project for CISC 594. The complete CISC-594 workspace is maintained in a GitHub repository, while the PantryPilot application remains a clearly bounded configuration-item directory. Approved baselines use main, controlled feature development, a test-before-merge workflow, and annotated release tags. A source-code ZIP may accompany the report for LMS submission, while the GitHub history provides the primary audit trail.

PantryPilot has two traceable baselines: v1.0.0 for the initial Plan & Adapt CM baseline and v1.1.0 for the controlled Safety & Grocery Controls update. The v1.1.0 update was developed on feature/safety-grocery-controls, tested, merged to main, and tagged after verification. The new Flask demonstration interface is documented in CHANGELOG.md under [Unreleased]; it exposes the v1.1.0 controls without changing the tagged baseline and must complete the same branch, test, review, and release process before a future tag.

Project Context and CM Objectives

PantryPilot is a smart meal and grocery planner that maintains dietary constraints, recipe data, pantry inventory, weekly plans, grocery lists, and AI-assisted substitutions. The local web application adds a presentation layer for demonstrating these controls through Plan, Grocery, Pantry, and Quality views. Because the project handles safety-sensitive constraints such as allergens and expiry dates, configuration management is needed to keep backend code, web assets, requirements, tests, configuration files, and release records synchronized.

The CM objective was to make every change traceable from a reason for change to branch work, test evidence, merge, release tag, and rollback point. This matches the assignment requirement to use version control, develop outside the main branch, test changes before merging, and tag completed software versions.

Repository and Tool Use

Table 1

Version-control evidence

CM item	Evidence	Purpose
Version-control tool	GitHub repository github.com/liangzixuan/cisc-594; PantryPilot files are in PantryPilot_CM_Generic_Files.	Provides auditable history, branch control, tags, and rollback.
Main branch	main	Approved working baseline after tested changes are merged.
Development branch	feature/safety-grocery-controls	Isolated new development from the baseline until verification.
Release tags	v1.0.0 and v1.1.0	Identify tested software versions and rollback points.
CI workflow	.github/workflows/pantrypilot-ci.yml	Runs pytest and the system-test runner for PantryPilot changes.
Instructor access	GitHub repository plus an LMS source ZIP when required.	Allows review of source, tests, branches, tags, workflow, and release records.

Configuration Items

The project treats backend and web source files, tests, static assets, runtime configuration, CM documentation, CI workflow, release notes, and version files as controlled configuration items. These items are tracked together because a release is only meaningful if the code, user interface, tests, configuration, and release description agree.

Table 2

Controlled configuration items

Configuration item	Location	Control method	Release relevance
Backend source	src/pantrypilot_app.py	Branch, commit, review, test	Implements deterministic release behavior.
Web application	src/web_app.py; templates/; static/	Branch, review, API tests, browser review	Presents release controls through the demo UI.
Tests	tests/test_smoke.py; tests/test_web_app.py	Required before merge	Verifies backend and HTTP integration behavior.
Runtime configuration	config/app_config.json	Version controlled	States baseline version and enabled features.
Release identity	VERSION, CHANGELOG.md	Updated with each release	Keeps version status accounting current.
Release notes	releases/	One file per tagged release	Documents scope, verification, and rollback.
Dependencies	requirements.txt	Pinned and reviewed	Controls Flask 3.1.1 and pytest 8.2.2.
CI workflow	.github/workflows/ pantrypilot-ci.yml	Protected repository file	Runs repeatable pytest and system checks.
CM documentation	docs/	Reviewed before baseline	Explains branching and change-control policy.

Formal Change-Control Process

The formal change-control process is intentionally lightweight but complete enough for the semester project. A change is not merged just because it compiles; it must have a stated purpose, a branch, test evidence, updated configuration records, and a rollback path.

Table 3

Change-control workflow

Step	Activity	PantryPilot evidence	Control value
1	Identify change request	Safety controls were required for v1.1.0; a local web demonstration was later requested for final presentation evidence.	Prevents unmanaged scope drift.
2	Create branch from main	feature/safety-grocery-controls	Keeps new development separate from baseline.
3	Implement and update configuration items	Backend source, Flask routes, template, JavaScript, CSS, local assets, tests, requirements, and changelog are controlled together.	Keeps executable behavior and records synchronized.
4	Test change set	11 pytest tests and 6 system scenarios passed locally; CI runs both commands.	Verifies backend and presentation-layer behavior before merge.
5	Merge to main	Merge commit 834e64e	Moves only tested change into approved baseline.
6	Tag release	Annotated tag v1.1.0	Creates a recoverable version reference.
7	Record status accounting	CHANGELOG.md and releases/release_notes_v1.1.0.md	Documents what changed and how to roll back.
8	Hold unreleased work	The Flask demo remains in CHANGELOG [Unreleased] until branch, review, merge, and tag criteria are complete.	Avoids silently redefining the v1.1.0 tag.

Branching, Merge, and Tagging Evidence

The repository history demonstrates the required branch workflow. The initial baseline was committed on main and tagged v1.0.0. New code was then developed on feature/safety-grocery-controls, committed as a branch change, merged back to main, and tagged v1.1.0 after verification.

Table 4

Repository history

Commit / tag	Branch	Description	CM meaning
842ae5e; tag v1.0.0	main	Establish PantryPilot CM baseline	Initial approved baseline.
72a997e	feature/safety-grocery-controls	Add safety and grocery control tests	Controlled branch implementation.
834e64e; tag v1.1.0	main	Merge safety and grocery controls	Merged tested release baseline.
51e7a65	main	Preserve PantryPilot project history in the full CISC-594 repository	Retains earlier branch and tag evidence after repository consolidation.

Commit graph evidence: full-repository main at 51e7a65; PantryPilot merge baseline 834e64e with tag v1.1.0; feature/safety-grocery-controls at 72a997e; original baseline 842ae5e with tag v1.0.0.

Testing and Release Verification

Testing is part of the CM gate. The GitHub Actions workflow installs requirements.txt, runs pytest, and executes system_test_runner.py for PantryPilot changes on pushes and pull requests to main. Local verification produced 11 passing pytest tests and 6 passing system scenarios. The five new web tests exercise the demo page, allergen decision endpoint, grocery aggregation endpoint, pantry boundary endpoint, and quality-summary endpoint.

The v1.1.0 tests cover release description, quality gates, allergen alias detection, safe recipe approval, grocery aggregation, and pantry-expiry boundary behavior. The added web/API regressions confirm that the interface delegates those decisions to the same deterministic backend controls. This evidence ties directly to unsafe-output, grocery-correctness, expiry-boundary, and UI/backend-consistency risks.

Baseline, Release, and Rollback Control

Table 5

Baseline and release records

Version	Release name	Tag	Verification	Rollback target
1.0.0	Plan & Adapt	v1.0.0	Initial source/config/test baseline	N/A
1.1.0	Safety & Grocery Controls	v1.1.0	6 release scenarios passed; CI workflow configured	v1.0.0
Unreleased	Web demonstration layer	No tag	11 pytest tests and 6 system scenarios passed locally	v1.1.0

Rollback is simple because tags identify recoverable baselines. If v1.1.0 fails release acceptance, the project can return to tag v1.0.0 while the feature branch is corrected and retested.

Repository Access and Source-Code Submission

The project is available in the full CISC-594 GitHub repository at github.com/liangzixuan/cisc-594. The PantryPilot directory, root-level workflow, branch history, annotated tags, tests, release notes, and CM documentation can therefore be inspected together. A separate source-code ZIP can be submitted through the LMS when the assignment requires file upload or an offline review copy.

No secrets are stored in the submitted source. Environment-specific values belong in config/environment.example or in local environment variables, not in committed files. The .gitignore excludes pycache files, pytest cache, virtual environments, logs, and .env files.

Conclusion

PantryPilot's CM process satisfies the assignment by using version control, isolating baseline development on a feature branch, testing before merge, tagging completed versions, and documenting status and rollback records. The web demonstration adds new controlled items and test evidence without changing the meaning of the v1.1.0 tag. Keeping that work under [Unreleased] until it completes the same review and release gates makes the repository auditable, reproducible, and recoverable.

References

Liang, Z. (2026a). PantryPilot configuration management generic files [Course assignment]. Harrisburg University of Science and Technology.

Liang, Z. (2026b). PantryPilot risk management report [Course assignment]. Harrisburg University of Science and Technology.

Liang, Z. (2026c). PantryPilot final project presentation [Course presentation]. Harrisburg University of Science and Technology.